

How the Current Code Works

Run Command

```
```bash
python3 main.py
```
```

High-Level Flow

1. `main.py` entrypoint starts (`if \_\_name\_\_ == "\_\_main\_\_":`).
2. It creates `docs\_majk\_djuk/` if missing.
3. It creates one timestamped log file per run (`run\_YYYYMMDD\_HHMMSS.txt`).
4. It mirrors stdout to both terminal and file via `Tee`.
5. It calls `main()` to execute the full experiment pipeline.

What `main()` Does

1. Loads datasets using `data\_loader.load\_datasets()`.
2. Adds time cyclic features using `preprocessing.add\_time\_cyclis\_features()`.
3. Prints dataset shapes.
4. Runs baseline experiment with all features via `train.run\_experiment(...)`.
5. Builds baseline feature list.
6. Runs leave-one-feature-out ablation via `train.run\_leave\_one\_feature\_out\_ablation(...)`.
7. Prints ablation table and summary (best drop, worst drop, negative deltas).
8. Runs multi-seed stability via `train.run\_seed\_stability(...)`.
9. Prints validation-only stability summaries and decision helper stats.

Core Training Logic (`train.run\_experiment`)

For each experiment:

1. Builds feature set as `all\_features - features\_to\_remove`.
2. Splits train/validation (and optionally test).
3. Fills NaNs using **train medians only**.
4. Fits `StandardScaler` on **train only** and applies to val/test.
5. Trains `MLPRegressor` with:
 - hidden layers `(256, 128, 64, 32)`
 - `activation="relu"`
 - `solver="adam"`
 - `max\_iter=1500`
 - seed from `random\_state`
6. Computes validation metrics (MAE, RMSE, R2).
7. Computes test metrics only if `evaluate\_test=True`.
8. Returns structured result dictionary.

Methodology Guardrails Preserved

- Validation set is used for feature/model selection.
- Test set is final checkpoint only.

- Multi-seed comparison is required.
- Ablation is validation-driven.
- Output is logged for reproducibility.

Files and Responsibilities

- `main.py` - orchestration and entry point
- `data\_loader.py` - CSV loading
- `preprocessing.py` - feature engineering
- `train.py` - experiment, ablation, and seed-stability execution
- `evaluate.py` - metric computation
- `utils.py` - helper utilities (`Tee`, parameter counting)